

VIBE CAFFÈ

Documentație Tehnică și Pedagogică

Proiect Web Full-Stack — Vibe Coding

Student: 2020nc

Data: 2026-04-03

Site live: vibe-website2.vercel.app

Repository: github.com/2020nc/vibe-website2

Commit-uri: 108 commit-uri Git (dec 2025 — apr 2026)

1. Introducere și Context

Proiectul "Vibe Caffè" este un site web complet pentru o cafenea de specialitate fictivă din București, dezvoltat în cadrul cursului Vibe Coding. Scopul proiectului este de a demonstra capacitatea de a construi o aplicație web modernă, de la zero până la producție, folosind tehnologii de nivel profesional.

Proiectul acoperă întregul ciclu de dezvoltare software: design, implementare front-end și back-end, integrarea cu o bază de date cloud, securizarea accesului administrativ și publicarea pe internet cu deployment automat.

1.1 Obiective Pedagogice

- Înțelegerea arhitecturii Next.js App Router (server vs. client components)
- Lucrul cu TypeScript în modul strict — tipuri, interfețe, tipizare API
- Integrarea Supabase ca backend-as-a-service
- Implementarea securității: hashing, cookie-uri httpOnly, middleware
- SEO tehnic: metadata, OpenGraph, JSON-LD structured data, SSR
- Deployment automat cu Vercel și Git (CI/CD)
- Versionare cod cu Git — 108 commit-uri cu mesaje descriptive

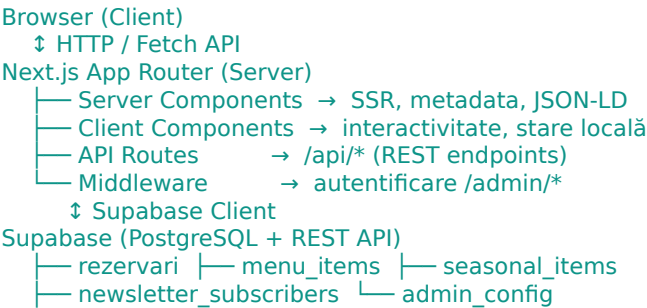
1.2 Metodologia Vibe Coding

Vibe Coding este o metodologie de dezvoltare colaborativă în care studentul dirijează procesul (decizii de design și arhitectură), iar un asistent AI (Claude) implementează codul. Această abordare accelerează învățarea prin expunerea la cod real de producție și discuții tehnice în timp real.

Notă: Documentația acoperă atât codul tehnic cât și procesul de gândire din spatele deciziilor de implementare.

2. Arhitectura Aplicației

2.1 Diagrama componentelor



2.2 Tipuri de componente

Tip	Caracteristici
Server Component	Randare pe server, fără JS în browser, bune pentru SEO, nu pot folosi useState
Client Component	Randare în browser, pot folosi stare și efecte, necesare pentru interactivitate
API Route	Endpoint REST pe server, accesibil via fetch
Middleware	Rulează înaintea oricărui request, folosit pentru autentificare

3. Stack Tehnic

Tehnologie	Versiune și rol
Next.js 15	Framework React — App Router, SSR, API Routes, Middleware
React 19	Biblioteca UI — hooks, componente funcționale
TypeScript 5	Tipizare statică (strict mode) — detectare erori la compilare
Tailwind CSS 4	Utilitare CSS — design responsiv, dark mode
Supabase	Backend-as-a-Service: PostgreSQL, REST API automat
Vercel	Hosting cloud cu deployment automat din GitHub (CI/CD)
Node.js crypto	Hashing SHA-256 pentru parole (built-in, fără dependențe)

4. Structura Proiectului

app/	
└─ layout.tsx	# Root layout — metadata globală, Navigation
└─ page.tsx	# Homepage SSR cu JSON-LD
└─ meniu/page.tsx	# Meniu cu filtrare categorii (Client)
└─ rezervari/page.tsx	# Formular rezervări + Supabase
└─ locatie/page.tsx	# Locație + JSON-LD LocalBusiness
└─ sarbatori/page.tsx	# Oferte sezoniere
└─ admin/page.tsx	# Panou admin (protejat de middleware)
└─ confidentialitate/	# Politică confidențialitate
└─ cookies/	# Politică cookies
└─ termeni/	# Termeni și condiții
└─ api/	
└─ admin/login/	# POST — autentificare
└─ admin/change-password/	# POST — schimbare parolă
└─ rezervari/	# GET/POST/PATCH — CRUD rezervări
└─ menu/	# GET/POST/PUT/DELETE — CRUD meniu
└─ holiday/	# GET/POST/DELETE — oferte sezoniere
└─ newsletter/	# POST — abonare newsletter
└─ chat/	# POST — Barista Bot AI
middleware.ts	# Protecție rute /admin/*
lib/supabase.ts	# Client Supabase singleton
public/robots.txt	# Instrucțiuni motoare de căutare

5. Pagini și Funcționalități

5.1 Homepage (/) — Server Component

Homepage-ul se randează complet pe server — motoarele de căutare primesc HTML complet. Aceasta este o decizie arhitecturală cheie pentru SEO.

- Hero section cu titlu, subtitlu și 2 butoane CTA (Meniu / Rezervă)
- Secțiunea "De ce Vibe?" — 4 carduri cu diferențiatori concreți
- Preview meniu SSR — 6 produse hardcodate (fără call la baza de date)
- Oferte sezoniere SSR — 3 produse cu descrieri
- JSON-LD schema.org CafeOrCoffeeShop pentru SEO local

Notă: Preview-ul meniu este hardcodat pentru a evita un call la DB pe homepage. Datele actualizabile sunt pe /meniu.

5.2 Meniu (/meniu) — Client Component

Client Component pentru că are nevoie de filtrare interactivă pe categorii, fără reîncărcare de pagină.

- Fetch la /api/menu la montare — date din Supabase
- Filtrare client-side: Espresso, Specialty, Vegan, Cold, Alternative, Pastry
- Metadata SEO în layout.tsx dedicat (Server Component)

Notă: Client Components nu pot exporta metadata. Soluție: layout.tsx (Server) exportă metadata, page.tsx (Client) gestionează interactivitatea.

5.3 Rezervări (/rezervari)

- Formular cu câmpuri: nume, telefon, dată, oră, persoane, mesaj
- POST la /api/rezervari → insert în tabelul Supabase
- Mesaj de succes personalizat cu buton înapoi
- Caseta informativă: program, confirmare, anulare

5.4 Panou Admin (/admin)

- Tab Rezervări: listare, confirmare, anulare
- Tab Meniu: adăugare, editare, ștergere produse
- Tab Sărbători: gestionare oferte sezoniere
- Tab Setări: schimbare parolă admin din panou

Notă: Navbar-ul site-ului este ascuns pe paginile /admin — interfață curată pentru administrator.

6. Baza de Date — Supabase

Schema tabelelor

rezervari

id BIGSERIAL PK | created_at TIMESTAMPTZ | nume TEXT | telefon TEXT
data DATE | ora TEXT | persoane INTEGER | mesaj TEXT
status TEXT DEFAULT 'pending' -- pending | confirmed | cancelled

menu_items

id BIGSERIAL PK | name TEXT | price INTEGER | category TEXT
description TEXT | ingredients TEXT | vegan BOOLEAN | available BOOLEAN

admin_config

id INTEGER PK DEFAULT 1 | password_hash TEXT -- SHA-256 hex (64 chars)

Notă: Un singur rând (id=1). Parola nu se stochează în clar — doar hash-ul SHA-256.

Client Supabase

```
export function getSupabase() {  
  return createClient(  
    process.env.NEXT_PUBLIC_SUPABASE_URL!,  
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!  
  );  
}
```

7. Securitate

7.1 Fluxul de autentificare

1. POST /api/admin/login cu { password }
2. Server: SHA-256(password) → hash
3. Compară cu admin_config.password_hash din Supabase
4. Match → setează cookie admin_token = hash (httpOnly, 8 ore)
5. Fiecare request /admin/* → middleware verifică cookie
6. Middleware: /^[a-f0-9]{64}\$/ .test(token) → format valid?

7.2 Codul de autentificare

```
function sha256(text: string): string {
  return createHash('sha256').update(text).digest('hex')
}
// Cookie httpOnly — inaccesibil din JavaScript browser
response.cookies.set('admin_token', hash, {
  httpOnly: true,
  secure: NODE_ENV === 'production', // Doar HTTPS
  sameSite: 'lax',                  // Protecție CSRF
  maxAge: 60 * 60 * 8,              // 8 ore
})
```

7.3 Principii de securitate aplicate

Principiu	Implementare
Hashing parole	SHA-256 — parola în clar nu se stochează niciodată
httpOnly cookie	Token inaccesibil din JS — protecție XSS
SameSite=Lax	Protecție împotriva CSRF
Secure flag	Cookie trimis doar pe HTTPS în producție
Variabile .env	Credențiale în .env.local, excluse din git
robots.txt	Blochează indexarea /admin de motoarele de căutare

Notă: În producție reală se recomandă bcrypt/argon2 în loc de SHA-256, pentru a evita atacuri brute-force.

8. SEO Tehnic

8.1 Sistem metadata Next.js

```
// layout.tsx — template global
export const metadata: Metadata = {
  metadataBase: new URL('https://vibe-website2.vercel.app'),
  title: {
    default: 'Vibe Caffè — Cafea de Specialitate în București',
    template: '%s | Vibe Caffè', // ex: "Meniu | Vibe Caffè"
  },
  openGraph: { siteName: 'Vibe Caffè', locale: 'ro_RO' },
}
```

8.2 JSON-LD Structured Data

```
{
  "@context": "https://schema.org",
  "@type": "CafeOrCoffeeShop",
  "name": "Vibe Caffè",
  "address": { "@type": "PostalAddress", ... },
  "openingHoursSpecification": [...]
}
```

Notă: JSON-LD ajută Google să afișeze "rich snippets" în rezultatele de căutare (adresă, orar, tip business).

8.3 robots.txt

```
User-agent: *
Allow: /
Disallow: /admin
Disallow: /api/
Sitemap: https://vibe-website2.vercel.app/sitemap.xml
```

9. Deployment și CI/CD

```
git push origin main
↓ webhook GitHub → Vercel
↓ npm run build (TypeScript check + Next.js build)
↓ deploy pe CDN global
→ vibe-website2.vercel.app (live în ~35 secunde)
```

Variabilă .env	Rol
NEXT_PUBLIC_SUPABASE_URL	URL proiect Supabase (public)
NEXT_PUBLIC_SUPABASE_ANON_KEY	Cheie anonimă Supabase (public)
ADMIN_SECRET	Parolă admin implicită (fallback)

10. Procesul de Învățare — Vibe Coding

10.1 Sprinturi de dezvoltare

Sprint	Obiectiv
Sprint 1	Fundație: setup Next.js, design system, homepage inițial
Sprint 2	Homepage SSR cu conținut real indexabil
Sprint 3	Meniu cu categorii și filtrare client-side
Sprint 4	Oferte sezoniere cu date din Supabase
Sprint 5	Locație + formular rezervări cu Supabase
Sprint 6	Navigare: active tracking, smooth scroll
Sprint 7	Pagini legale + robots.txt
Modulul 4 P1-P7	SEO avansat, admin securizat, brand unificat
Extra	Schimbare parolă admin fără email recovery

10.2 Concepte cheie asimilate

Server vs. Client Components

Decizia de a face o componentă server sau client are implicații directe asupra SEO-ului. Server = rapid, bun pentru SEO, fără stare locală. Client = interacție direct la SEO.

Cookie httpOnly vs. localStorage

Pentru autentificare s-a ales cookie httpOnly în loc de localStorage. Motivul: localStorage este accesibil din JavaScript (vulnerabil XSS). Cookie httpOnly nu poate fi citit de cod JS malițios.

Hashing parole (SHA-256)

Parola nu se stochează în clar — doar hash-ul SHA-256 (funcție one-way). Din hash nu se poate reconstitui parola originală. Practică standard de securitate.

Metadata în Client Components

Client Components nu pot exporta metadata. Soluție arhitecturală: se creează un component separat (Server Component) care exportă metadata, permițând page.tsx să importe acest component.

11. Concluzii

11.1 Realizări tehnice

- Aplicație web full-stack completă, publicată și funcțională
- Arhitectură modernă Next.js 15 cu separare corectă server/client
- Securitate implementată corect: hashing, httpOnly cookies, middleware
- SEO tehnic complet: metadata, OpenGraph, JSON-LD, robots.txt
- Bază de date cloud cu 5 tabele și operații CRUD complete
- Deployment automat CI/CD (GitHub → Vercel, ~35 secunde)
- 108 commit-uri Git cu mesaje descriptive pe 4 luni de dezvoltare

11.2 Lecții cheie

- Arhitectura contează: deciziile timpurii au impact pe termen lung
- Securitatea nu este opțională — chiar și pentru un site simplu
- SEO tehnic este parte din cod, nu un afterthought
- Variabilele de mediu sunt esențiale pentru separarea config de cod
- Git disciplinat face debugging-ul mai ușor și colaborarea posibilă

11.3 Îmbunătățiri viitoare

- Înlocuirea SHA-256 cu bcrypt pentru hashing mai robust
- Sistem de email pentru confirmare rezervări și recuperare parolă
- Dashboard analytics pentru admin
- Teste automate (unit + integration tests)
- Optimizarea imaginilor cu next/image